

Code Walkthrough

Tango - Personal Reference Guide

Version 0.4.0 | July 2026

This document explains what each major code block in the Tango pipeline does and why it was written the way it was. It is intended as a personal reference to help you understand the code at a fundamental level, not as an API reference.

config.py

This file is the first thing loaded when the pipeline starts. It calls `load_dotenv()` which reads your `.env` file and sets all the variables as environment variables. Then it reads each variable with `os.getenv()` and assigns them to Python constants.

The reason every other module imports from config rather than calling `os.getenv()` directly is testability. In tests, you can change a constant in config without touching environment variables. It also means there is exactly one place to look if you want to know what value a setting has.

The `MODEL_ID` and `DECK_ID` constants deserve special attention. `genanki` uses these numbers to identify card models and decks across Anki imports. If you change them, Anki will treat all your existing cards as belonging to a different model, which breaks your review history. They are intentionally hardcoded with a comment warning against changing them.

language.py

`LANGUAGE_MAP` is a Python dictionary where every key is a lowercase language name and every value is a BCP-47 code like `fr` or `zh-CN`. There are multiple keys per language because people name their decks differently. French might be French, Francais, Deutsch speakers call it Französisch, and so on.

`resolve_language_code(language_flag, deck_name)`

This is the main function other modules call. It first checks if the `--language` flag was passed. If it was, that value wins immediately. If not, it calls `_infer_from_deck_name()`. If that also returns nothing, it raises `LanguageResolutionError` with a helpful message telling the user exactly what to do.

`_infer_from_deck_name(deck_name)`

This function strips the `::` sub-deck separators, lowercases everything, and checks each word against `LANGUAGE_MAP`. So `Language::French::B2` becomes `language french b2`, and `french` is found in the map. Multi-word language names like `Traditional Chinese` are

checked separately as phrases.

resolve_transcript(transcript_list, language_code)

youtube-transcript-api uses exact key lookup internally, so passing fr will not match fr-FR. This function adds partial matching by iterating over all available transcripts and finding any whose language code starts with your requested code. Among partial matches, it returns manual transcripts before auto-generated ones.

transcript.py

_build_proxy()

Reads WEBSHARE_USERNAME and WEBSHARE_PASSWORD from config. If both are set, returns a WebshareProxyConfig object. If only generic proxy URLs are set, returns a GenericProxyConfig. If neither is set, returns None. The YouTubeTranscriptApi constructor accepts None for no proxy, so this works cleanly.

get_transcript(video_id, languages)

Creates a YouTubeTranscriptApi instance with the proxy config, calls api.list(video_id) to get all available transcripts, then passes the transcript list to language.resolve_transcript() to pick the best one. Every error from the library is caught and re-raised as a more descriptive typed exception.

get_snippets(transcript)

Calls transcript.fetch() to get the actual text data. Iterates over the snippets, cleans each one with _clean(), and builds a dictionary keyed by timestamp floats. Also stores the full joined text under the _full_text key. The timestamp keys are how you find which part of the video a word came from, which matters for the transcript example sentence lookup later.

_clean(text)

Decodes HTML entities like & into real characters, removes annotation tags like [Music] or [Applause] using a regex, and collapses whitespace. This is important because auto-generated transcripts are full of these noise patterns.

nlp.py

Lazy loading (_get_model())

The spaCy model is not loaded when you import nlp.py. It is only loaded the first time process_transcript() is called. This matters because loading the model takes about a second and importing it at module level would slow down every test that imports nlp.py, even tests that never process any text.

`process_transcript(text)`

Passes the full transcript text to the loaded spaCy model. The model returns a Doc object, which is an iterable of Token objects. The function iterates over tokens, calls `_is_valid_token()` on each one, and if valid, uses `token.lemma_.lower()` as the dictionary key. The first time a lemma appears, it gets a frequency of 1. Each subsequent appearance increments the count. The insertion order of a Python dict is guaranteed since Python 3.7, which is how first-appearance ordering is preserved.

`_is_valid_token(token)`

Returns True only if `token.is_alpha` is True and `token.pos_` is in the set NOUN, VERB, ADJ, ADV. `is_alpha` filters out punctuation, numbers, URLs, and symbols. The POS filter removes prepositions, conjunctions, articles, and other grammar words that are unlikely to be useful vocabulary. Stop words are intentionally kept because beginners need basic words too.

`deck.py`

`_anki_request(action, **params)`

Every AnkiConnect call goes through this function. It builds the JSON payload in the format AnkiConnect expects, posts it to ANKI_HOST, checks for errors in the response, and returns the result field. If the connection is refused or times out, it raises `AnkiNotRunningError`. If AnkiConnect returns an error string, it raises `AnkiConnectError`.

`get_card_fronts(deck_name)`

First calls `findNotes` to get all note IDs in the deck, then calls `notesInfo` to get the field data for all those notes. Extracts the Front field from each note, lowercases it, and returns the list. The two-step process is necessary because AnkiConnect does not support fetching field data in a single query by deck name.

`_is_sentence_structured_deck(fronts)`

Counts the average number of words per card front. If the average is above 3.0, the deck is considered sentence-structured. This detects decks like grammar study decks where fronts are questions or example sentences rather than single words. Fuzzy matching is meaningless against sentence fronts because short lemmas will always find themselves as substrings of unrelated sentences.

`_check_single(lemma, fronts, skip_fuzzy)`

Runs the confidence interval check for one word. First checks for exact match, which works regardless of deck structure. If `skip_fuzzy` is True, returns NEW for anything that is not an exact match. Otherwise, filters out short fronts from the candidate pool, runs `WRatio` to find the best match, then applies `token_sort_ratio` and `length_ratio` as secondary filters. All three

conditions must pass to classify as QUEUE.

prompt_queue(queue)

Displays each queued word and its closest deck match in the terminal. Reads y, n, or s from the user. y adds the word to the approved list. n defers it. s defers the current word and all remaining words to review.json without prompting further. The user choice for each run is stored in _user_choice so the prompt only appears once per language pair.

definition.py

Dual-source strategy in fetch_definition()

The function always makes a native-language call first to `dictionaryapi.dev/{language}/` regardless of what DEF_LANG is set to. This collects example sentences, synonyms, and antonyms in the original language. Then it resolves whether to translate the lemma and which source to use for the definition. This separation means example sentences are always in the language the user is learning, while the definition explanation can be in whatever language helps them understand.

Translation block

If `def_language` differs from `language`, `translate_word()` is called to convert the lemma into the target language before fetching the definition. For example, if the video is French and DEF_LANG is en, `bonjour` is translated to `hello` and then the English definition of `hello` is fetched from MW. The translated word is only used for the definition lookup, not for the card front which always shows the original lemma.

_parse_mw_response(lemma, data, snippets)

The MW API returns a complex nested structure. The definition is in `shortdef` which is a clean list. The examples are buried inside `def -> sseq -> sense -> dt -> vis`, which stands for verbal illustrations. This function navigates that structure and collects up to two examples. Synonyms come from a `syns` field that is not always present. All MW-specific markup tokens like `{bc}` and `{it}` are stripped by `_strip_mw_markup()`.

_cap(text)

A local helper inside `fetch_definition` that truncates text to 256 characters. It tries to cut at the last period before the limit rather than cutting mid-sentence. This applies to every text field before the `DefinitionResult` is built.

SQLite caching

The cache key is a composite of the lemma and the target language, like `contaminate::en`. This means the same word can be cached once for English definitions and once for French definitions. The `_get_db()` function creates the table if it does not exist and runs an ALTER

TABLE migration to add `example_dict2` if the database was created before that column existed.

translation.py

Three-tier resolution in `translate_word()`

First tries community LibreTranslate mirrors by calling `_probe_mirror()` and then `_translate_via_mirror()`. If all mirrors are down, tries `translate_local()` which uses the locally installed argostranslate model. If the model is not installed, enters the user prompt flow. The prompt only fires once per language pair per run because the user choice is stored in `_user_choice`.

ThreadPoolExecutor timeout in `translate_local()`

argostranslate on CPU without GPU acceleration can take 30 to 60 seconds per word when loading the Stanza sentence boundary detection model. The function wraps the translation call in a ThreadPoolExecutor and sets a timeout of `LOCAL_TRANSLATION_TIMEOUT` seconds. If the translation takes longer than that, it returns None and the pipeline creates a fallback card. This prevents the pipeline from appearing frozen.

`download_model(from_code, to_code)`

Updates the argostranslate package index, finds the download URL for the requested language pair, and streams the download using requests with `stream=True`. The streaming is what makes the progress bar possible because it receives data in chunks of 8192 bytes and can track how many bytes have arrived relative to the Content-Length header. After download, `install_from_path()` installs the model permanently.

cards.py

`_build_model()`

Creates a `genanki.Model` with the fixed `MODEL_ID` constant and ten named fields. The model also contains the card template HTML and CSS. The template uses Anki's conditional tags like `{{#Synonyms}}` to hide sections when fields are empty. The CSS uses Anki's `var(--fg)` and `var(--canvas)` CSS variables which automatically adapt to the user's light or dark mode theme.

`_build_note(result, model, video_id)`

Creates one `genanki.Note` from a `DefinitionResult` object. The GUID is computed from the lemma and video ID using `genanki.guid_for()`. This means if you run the pipeline on the same video twice, the GUIDs are identical and Anki will silently skip the duplicates on import rather than creating double cards. The fields list must exactly match the order declared in `_build_model()` or the fields will appear on the wrong parts of the card.

`_truncate(text, max_chars)`

Truncates text at a sentence boundary rather than mid-word. It finds the last period followed by a space before the character limit. If that point is in the second half of the allowed length, it cuts there. Otherwise it does a hard cut and appends an ellipsis. This is called on synonyms and antonym HTML before it goes into the card fields.

`build_package(video_id, deck_name, found, not_found, snippets)`

The main output function. Iterates over DefinitionResult objects to build standard cards. For words in not_found, searches the snippets dictionary for a sentence containing the word to use as the only example. Writes a .apkg file to the OUTPUT_DIR with a timestamped filename. Returns a PackageResult dataclass with the path and accurate counts of standard cards, fallback cards, and skipped words.

state.py

SQLite tables owned by state.py

processed_videos records each completed pipeline run with the video ID, timestamp, deck, and card count. generated_packages logs every .apkg file path for reference. vocabulary stores every lemma extracted from every video with its frequency and position. The vocabulary table is the foundation for Phase 3 data science features because it records the word, which video it came from, how often it appeared, and where in the transcript it first appeared.

Session class

A simple in-memory container for the user's selected deck. It does not write to SQLite. When the process exits, the session is gone. This is intentional because deck selection is expected to be explicit each run rather than remembered across sessions. Future versions may add a preference system for default deck names.

`_now()`

Returns the current UTC time as an ISO 8601 string using datetime.now(timezone.utc). Using timezone-aware datetimes prevents bugs when the user's system clock is in a different timezone. The older datetime.utcnow() is deprecated in Python 3.12 which is why this form is used.

`__main__.py`

Argument parser

Uses Python's built-in argparse module. The --review and --process-backlog flags are in a mutually exclusive group, meaning the user cannot pass both at once. The --language and --def-lang flags are optional strings. The --verbose flag sets the logging level to DEBUG so

that every internal log message appears in the terminal.

`_run_pipeline(args, session)`

The main pipeline orchestrator. Calls each module in the correct order: language resolution, video duplicate check, transcript extraction, NLP, deck check, CLI prompt for queued words, definition fetching, card generation, state logging. If any step fails with an expected error, it prints a formatted error message and exits with code 1. The language code and definition language are resolved before any API calls so the user sees them confirmed at the top of the output.

`_prompt_import(apkg_path)`

Asks the user if they want to import the .apkg into Anki automatically. If yes, sends an `importPackage` request to AnkiConnect with the absolute path of the file. WSL users may see this fail because the Linux path `/mnt/c/...` is not recognised by Windows AnkiConnect. This is a known limitation and manual import via File -> Import in Anki is the workaround.

`reset_warning_state()` call at startup

`translation.py` tracks which warnings have been shown and which user choices have been recorded in module-level dictionaries. These must be cleared at the start of each pipeline run so that a second run in the same Python process does not skip warnings that should appear again. `_run_pipeline()` calls `reset_warning_state()` before the definition fetching step.